

UniWeb — Developer Specification

11 Feature Points · Correct Implementation · Do / Don't · Multi-GST · Full E-Mandate Channels

Uniweb Technologist PVT LTD · Brand: UniWeb · 06 August 2026 · v1.0

How to use this document

- Every point has: Goal · Why · Correct way (Do) · Wrong way (Don't) · Technical steps · Acceptance.
- Points 1–6 = original requirements. Points 7–11 = mandatory for safe P2M live + compliance.
- Implement in priority order at end. Do not skip 7/8 before live money.

Contents

- 1. Real Client IP + Live Geolocation on KYC/Upload
- 2. Zero-Touch Automated KYC & Business Verification
- 3. Dynamic CMS + Dynamic MDR / Pricing Engine
- 4. PAN vs Multiple GSTIN — Smart Duplicate Logic (Multi-Business)
- 5. Multiple P2M QR Codes (Fixed Amount + Open Amount)
- 6. E-Mandate / UPI Autopay / Recurring (ALL channels, not UPI-only)
- 7. Webhook Idempotency + Ledger Lock (money safety)
- 8. Verified Beneficiary + Auto Payout (replace Manual UTR)
- 9. Risk / Velocity Rules after Auto-KYC
- 10. Audit Trail + Evidence Pack
- 11. Partner Forward Payload after Auto-Approve
- Appendix: Priority order + Acceptance checklist

Point 1 — Real Client IP + Live Geolocation (KYC / Upload)

Goal: Every KYC action and document upload must store the real end-user IP and the device's live GPS coordinates at that moment — not Cloudflare/proxy IP and not coarse IP-geo only.

Why: Partner / bank / compliance audits ask “where was this person when they uploaded Aadhaar?”. Wrong IP/location = failed audits and fraud disputes you cannot defend.

DO (■■■■ ■■■■) ✓ Read X-Forwarded-For / CF-Connecting-IP / True-Client-IP; take left-most public IP. ✓ On upload/KYC page: request HTML5 geolocation (user gesture); store lat, lng, accuracy_m, timestamp. ✓ Store geo_source = html5 ip_fallback; if user denies, mark location_status = denied and still allow with IP-only + flag. ✓ Bind IP + geo to the same kyc_event_id / upload_id row.	DON'T (■■■■ ■■ ■■■■) ✗ Trust REMOTE_ADDR alone behind Cloudflare (you will get CF edge IP). ✗ Use only MaxMind IP-geo as “exact location” — it is city-level, not device GPS. ✗ Silently skip geo without recording that user denied permission. ✗ Allow KYC complete with empty IP field.
---	---

Technical steps

- Server helper getClientIp(request): parse X-Forwarded-For chain; validate public IP; store ip_raw + ip_resolved.
- Frontend: navigator.geolocation.getCurrentPosition on “Start Video KYC” / “Upload document” click (must be user gesture).
- POST body includes: latitude, longitude, accuracy, geo_ts, geo_permission (granted|denied|unavailable).
- DB columns on kyc_events / document_uploads: client_ip, ip_country, lat, lng, geo_accuracy_m, geo_source, user_agent, device_fingerprint.
- Admin KYC detail screen shows map pin + IP + accuracy; export in evidence pack (Point 10).

Acceptance: Behind Cloudflare, stored IP = user's real public IP. With GPS on, lat/lng within ~50m of test location. Deny path still saves IP and flags geo denied.

Point 2 — Zero-Touch Automated KYC & Business Verification

Goal: PAN / Aadhaar / GST / MCA verified via government-connected APIs (Karza / Digio / Signzy etc.). Face match + name match. If all pass → auto Approved and ready to push partner — no admin file opening.

Why: Manual admin KYC does not scale and creates bottleneck. Partners expect verified digital KYC, not PDF folders.

DO (■■■■ ■■■■) ✓ API-fetch PAN/GST/Aadhaar where legally allowed; auto-fill profile fields. ✓ Liveness check on selfie/video + face match vs ID photo; configurable threshold (e.g. ≥ 88). ✓ Name match: individual KYC name vs GST legal name / MCA directors (fuzzy + normalized). ✓ Pvt Ltd / LLP: MCA CIN / director verify before auto-approve. ✓ On fail: merchant sees clear “Fix & retry”; after N fails → risk queue (not silent admin pile). ✓ Store every API request id + response hash for audit.	DON'T (■■■■ ■■ ■■■■) ✗ Hard-code 100% pixel face match — genuine users will fail; use score thresholds. ✗ Auto-approve with only document upload and no API verify. ✗ Skip MCA for companies. ✗ Let merchant proceed to live collect while status = pending_review without limit.
---	--

Technical steps

- Integrations module: providers for PAN, GSTIN, Aadhaar (OTP/DigiLocker), Face, MCA — swap by config.
- Onboarding wizard steps: Entity type → API verifies → Face/liveness → Name match engine → Decision.
- Decision engine: if all required checks = pass → status=approved, approved_at, approved_by=system.
- If any fail → status=action_required + reason codes; block live keys until fixed or manual override.
- Optional: after approved, trigger Point 11 partner forward.
- Cost control: cache GST/PAN verify 24h; rate-limit per merchant.

Acceptance: Happy-path merchant reaches Approved with zero admin click. Fail path never gets live collect. All API refs visible in admin evidence.

Point 3 — Dynamic CMS + Dynamic MDR / Pricing Engine

Goal: Admin toggles menus/modules on Website, Customer, Merchant, Sub-merchant, Staff without code deploy. MDR set globally and per-merchant; public pricing page reads from same config.

DO (■■■■ ■■■■) ✓ Feature flags table: key, scope (global role merchant), enabled, updated_by, updated_at. ✓ MDR: global base + merchant override (absolute or delta); effective_from datetime. ✓ Admin matrix: Partner gateway cost MDR vs Platform charge MDR → margin column. ✓ Public /pricing page fetches published rates API (cache with short TTL + purge on save). ✓ Audit every MDR change (old → new).	DON'T (■■■■ ■■ ■■■■) ✗ Hard-code menu items in PHP templates with no admin switch. ✗ Change live MDR with no effective_from (mid-day chaos on in-flight txns). ✗ Show partner cost to merchants. ✗ Deploy code to change homepage section visibility.
---	--

Technical steps

- Tables: feature_flags, mdr_rules (scope, method, rate_bps, effective_from, merchant_id nullable).
- Resolver: getMdr(merchant_id, method, at_time) → override else global.
- Admin UI: toggles + MDR form + margin matrix read-only from partner_cost config.
- Website pricing: only rows marked is_public=1.

Point 4 — PAN vs Multiple GSTIN (Multi-Business Register Logic)

Goal: Do not blindly block duplicate PAN. Support one PAN → many GSTINs (branches / states / entities) under one login with business switcher. Block only true duplicates.

Why: Real businesses (Pvt Ltd, multi-state proprietorship) operate multiple GSTINs on one PAN. Blind unique(PAN) kills onboarding and looks unprofessional.

Validation matrix (implement exactly)

Scenario	PAN	GSTIN	Action	User message
1 Individual / unregistered exact duplicate	Exists	Empty / same empty	BLOCK signup	An account with this PAN already exists. Please log in.
2 Same PAN + same GSTIN	Exists	Same GSTIN exists	BLOCK	This Business/GSTIN is already registered with us.
3 Same PAN + NEW different GSTIN	Exists	New valid GSTIN	ALLOW	Continue onboarding. You can link under existing login.
4 New PAN + any GSTIN	New	Any valid	ALLOW	Normal signup

GSTIN rules (must code)

- GSTIN length 15; positions 3–12 must equal PAN (uppercase). If not → reject format before DB check.
- State code (first 2 chars) + entity checksum validated with official checksum algorithm.
- Normalize PAN/GSTIN to uppercase, strip spaces before compare.

Data model (Multi-tenant / Multi-business)

- users (login identity): id, email, phone, password_hash — one human login.
- merchants (business entity): id, pan, gstin (nullable unique with pan), legal_name, kyc_status, parent_user_id.
- UNIQUE constraint: (pan, gstin) where gstin is not null; partial unique on pan where gstin is null (only one non-GST account per PAN).
- user_merchant_roles: user_id, merchant_id, role — enables one user many merchants.
- Session: active_merchant_id; UI switcher lists all merchants for user.

DO (■■■■ ■■■)

- ✓ On Scenario 3: after GSTIN verify API, create new merchant row linked to same user (or invite link to claim).
- ✓ Business switcher in header: switch active_merchant_id; all dashboards filter by it.
- ✓ KYC / settlement / QR scoped to active merchant only.
- ✓ Admin can see all merchants under same PAN as a group.

DON'T (■■■■ ■■ ■■■)

- ✗ UNIQUE index on pan alone.
- ✗ Force second password for second GSTIN.
- ✗ Mix transactions of two GSTINs in one ledger without merchant_id.
- ✗ Allow two rows with same pan and null gstin.

Acceptance: Same PAN + two different valid GSTINs → two merchant profiles, one login, switcher works. Same PAN+GSTIN twice → blocked. Same PAN twice without GST → blocked.

Point 5 — Multiple P2M QR (Fixed + Open Amount)

Goal: Merchant creates many P2M QRs with labels; each is either Fixed amount (locked) or Open amount; reports split by QR.

DO (■■■■ ■■■)

- ✓ Table merchant_qrs: id, merchant_id, label, amount_mode (fixed|open), amount, partner_ref, status (active|disabled), created_at.
- ✓ Fixed: UPI intent / QR with am= parameter; customer cannot edit amount.
- ✓ Open: no am= ; customer enters amount in UPI app.
- ✓ Every payment stores qr_id; reports group by label.
- ✓ Disable QR without deleting history.

DON'T (■■■■ ■■ ■■■)

- ✗ One QR only per merchant.
- ✗ Reuse same partner QR string for different labels without mapping table.
- ✗ Allow amount edit on Fixed QR.
- ✗ Delete QR rows that have payments (soft-disable only).

Technical steps

- Create UI: Label, Fixed/Open toggle, amount if fixed, optional expiry.
- Call partner static/dynamic QR API; store image URL + payload + qr_id.
- Webhook → resolve qr_id → credit that merchant + tag transaction.qr_id.
- Merchant report: filter by QR label; export Counter1 vs Delivery etc.

Point 6 — E-Mandate / Recurring (ALL standard channels, not UPI-only)

Goal: Merchants create subscription plans and collect recurring payments via the full mandate stack supported in India — not only UPI Autopay.

Why: Many customers still mandate via Net Banking / Debit Card / Aadhaar eSign (e-NACH). UPI Autopay alone leaves money on the table and is not “professional full stack”.

Mandatory channels to support (via partner)

Channel	Typical use	Notes
UPI Autopay	Retail subscriptions, low-mid ticket	NPCI rules; amount caps; app approval
e-NACH / e-Mandate (Net Banking)	EMI, higher ticket, traditional banks	Bank account debit via NACH
Debit Card mandate	Where issuer supports	Through PA / billdesk-style rails
Aadhaar eSign e-NACH	When customer has no netbanking comfort	Aadhaar OTP eSign on mandate form
Credit Card standing instruction	Where partner + issuer allow	Often via card network SI; PCI scope
Wallet Autopay (if partner offers)	Closed/semi-closed wallets	Only if licensed partner exposes API

Platform must expose channel as a selectable method on mandate registration link. Actual debit rails = partner APIs (Cashfree Sub, Razorpay Subscriptions, Digio eSign+NACH, etc.).

DO (■■■■ ■■■)

- ✓ Plan entity: amount, frequency (weekly/monthly), billing_day, cycles (or until cancelled), max_amount.
- ✓ Registration link selects channel → partner mandate create → webhook status active/pending/failed.
- ✓ Cron on due date: create debit request per active mandate; idempotent debit_id.
- ✓ Bounce: map reason, notify customer+merchant, retry policy (max 3), then pause.
- ✓ Merchant UI: Active / Pending / Bounced / Completed / Cancelled; Pause / Cancel.

DON'T (■■■■ ■■ ■■■)

- ✗ Build only UPI Autopay and claim “full recurring”.
- ✗ Debit without storing mandate_id + partner_ref.
- ✗ Retry forever on insufficient funds.
- ✗ Ignore NPCI / NACH debit notification windows.

Technical steps

- Tables: subscription_plans, mandates (channel, partner_mandate_id, status, max_amount), mandate_debits (due_at, status, utr, fail_reason).
- Adapter per channel family: UpiAutopayAdapter, EnachAdapter, CardSIAdapter — same interface.
- Scheduler: every hour pick due debits next 24h; enqueue; worker calls partner; webhook completes.
- Ledger: successful debit → same capture path as one-time P2M (commission + available).

Acceptance: Merchant can create plan and registration link for at least UPI Autopay + e-NACH. Failed debit shows mapped reason. Cancel stops future cron debits.

Point 7 — Webhook Idempotency + Ledger Lock

What this is: Guarantee that one payment success event credits the merchant **exactly once**, even if partner retries webhook 10 times. Ledger moves through strict states so settlement cannot double-pay.

Why P2M live breaks without it: Partners retry webhooks. Without idempotency you double wallet credit → you lose money or dispute with merchant. Without ledger lock, two settle clicks pay twice.

DO (■■■■ ■■■)

- ✓ webhook_events.event_id UNIQUE; INSERT first; only winner processes credit.
- ✓ Return 200 on duplicate immediately.
- ✓ Ledger states: available ↔ in_transit ↔ settled | reversed; one DB transaction per transition.
- ✓ Settle button creates payout_job only if available >= amount (row lock merchant wallet).

DON'T (■■■■ ■■ ■■■)

- ✗ Credit on every webhook POST without checking event_id.
- ✗ UPDATE balance with free-form SQL outside ledger_entries.
- ✗ Allow parallel settle requests without lock.

Technical steps

- Tables: webhook_events, ledger_entries, merchant_balances (or computed from entries).
- Process: verify signature → claim event → insert ledger capture → update balance → mark processed.
- Tests: replay same payload 5 times → one credit.

Point 8 — Verified Beneficiary + Auto Payout (Replace Manual UTR)

What this is: Merchant adds bank/UPI → system verifies account (penny drop / name match) → settlement can auto-push via IMPS/NEFT/UPI Payout API → UTR comes from partner webhook. Manual UTR typing becomes emergency-only.

Why: Manual UTR does not scale, causes wrong UTR/amount errors, and looks unprofessional to merchants who expect Cashfree/Razorpay-like “Settle now”.

DO (■■■■ ■■■■) ✓ merchant_beneficiaries with status unverified verified failed. ✓ Only verified selectable for auto payout; unverified → Manual UTR path only. ✓ payout_jobs lifecycle + ApiPayoutAdapter + ManualUTRAAdapter (same UI). ✓ Payout fail → reverse in_transit to available + mapped reason. ✓ Maker-checker above threshold (e.g. ■1,00,000).	DON'T (■■■■ ■■ ■■■■) ✗ Auto-payout to never-verified account numbers. ✗ Delete beneficiary history after payout. ✗ Leave money in in_transit on fail with no reverse.
--	--

Keys: Platform adapters + UI can be built with mocks now. Live penny-drop + payout need partner/bank keys later (Phase D).

Point 9 — Risk / Velocity Rules (After Auto-KYC)

What this is: Even after Zero-Touch KYC approve, automatically hold or flag merchants when behaviour looks risky (velocity, amount spike, device reuse) — without waiting for admin to notice.

Why: Auto-KYC removes human review. Fraudsters will pass API checks. Velocity rules are the safety net before large settlements leave the platform.

DO (■■■■ ■■■■) ✓ Rules engine: e.g. >N txns/hour, first-day volume >X, same device many PANs, sudden 10x average ticket. ✓ Actions: flag hold_settlement block_collect (configurable). ✓ Hold = available → hold balance; admin release with note. ✓ All rule hits in risk_events for audit.	DON'T (■■■■ ■■ ■■■■) ✗ Auto-approve KYC and allow unlimited same-day settlement with no velocity check. ✗ Permanent ban on first mild rule hit — prefer hold + review.
---	---

Point 10 — Audit Trail + Evidence Pack

What this is: Immutable log of KYC and money actions + one-click export package (IP, geo, face score, API refs, ledger moves, UTRs) for partner, bank, or dispute.

Why: When a chargeback or regulatory question comes, “check the admin panel” is not an answer. Evidence pack is the professional response.

DO (■■■■ ■■■■) ✓ audit_log append-only: actor, action, entity_type, entity_id, before_json, after_json, ip, at. ✓ KYC evidence: Point 1 fields + API verification ids + face score + decision. ✓ Money evidence: ledger_entries + payout_jobs + webhook event ids. ✓ Admin button: Download Evidence Pack (ZIP/PDF) for merchant_id or txn_id.	DON'T (■■■■ ■■ ■■■■) ✗ Overwrite KYC rows without history. ✗ Allow ops to edit UTR without audit row.
---	--

Point 11 — Partner Forward Payload (Post Auto-Approve)

What this is: After merchant is system-approved, UniWeb can send structured onboarding payload to one or many payment partners (status matrix per gateway). Not raw document email spam.

Why: You will use multiple PAs/banks. Manual re-upload of KYC to each portal does not scale. One-click / auto forward with per-partner status is how serious platforms operate.

DO (■■■■ ■■■■) ✓ gateway_submissions: merchant_id, gateway, status (pending/approved/rejected), external_ref, last_error. ✓ Forward verified fields + secure document links (time-limited), not unencrypted email attachments by default. ✓ Admin matrix UI: merchant × gateway statuses; retry button. ✓ Notify merchant when a gateway approves/rejects.	DON'T (■■■■ ■■ ■■■■) ✗ Email full Aadhaar PDF to 5 partner inboxes as the only integration path. ✗ Mark merchant live on UniWeb when partner still rejected.
---	---

Appendix A — Implementation Priority

Priority	Points	Rationale
P0 — Before live money	1, 4, 5, 7, 8	Compliance evidence, multi-business, P2M QR, no double credit, settlement path
P1 — Scale & control	2, 3, 9, 10	Auto KYC, MDR/CMS, risk holds, evidence pack
P2 — Growth	6, 11	Full recurring channels, multi-gateway forward

Appendix B — Global Acceptance Checklist

- CF/proxy: KYC row shows real client IP + optional HTML5 lat/lng with accuracy.
- PAN+two GSTINs → one login, two merchants, switcher; duplicate PAN+GST blocked.
- Merchant creates 3 QRs (2 fixed 1 open) with labels; report splits correctly.
- Same webhook event replayed → single ledger credit.
- Settle → verified beneficiary → payout_job success path (mock OK) updates UTR; fail reverses balance.
- MDR change in admin reflects on public pricing within cache TTL; margin matrix visible only to admin.
- Mandate registration offers UPI Autopay and at least one e-NACH/NetBanking path in UI (partner-backed).
- Evidence pack download contains KYC + payment + payout trail for a test merchant.

Final note for developers

- Do not implement “happy path only”. Every money and KYC path needs fail + retry + audit.
- Mock partner adapters in dev; same interfaces used in production with real keys.
- When in doubt: strict ledger + idempotency first, features second.